

```
import pandas as pd
from requests import get
```

```
data = {}
response = get("https://requests.?!/compare/...")
data = json.loads(response.text)
df = pd.DataFrame(data)
df.dropna()
df['date'] = pd.to_datetime(df['date'])
cleaned_data = df.reset_index(drop=True)
```

# APIs & Data Wrangling

**Domando los datos del mundo real  
para la Ciencia de Datos.**

```
import pandas as pd
from requests import get
data = json.loads(response.text)
df = pd.DataFrame(data)
df.dropna()
df['date'] = pd.to_datetime(df['date'])
cleaned_data = df.reset_index(drop=True)
```

```
for item in api_response:
    process(item)
```



# Roadmap de hoy

Los check-points de nuestra clase.



## 1. APIs 101

¿Qué son y cómo hacen plata las empresas con ellas?



## 2. Data Wrangling

El arte de limpiar el caos (y el 60% de tu trabajo).



## 3. Pipeline de 6 pasos

Del descubrimiento a la publicación.



## 4. Pandas en acción

`merge()` vs `concat()`.



# APIs: El mozo del restaurante 🍴

Application Programming Interface en lenguaje humano.

Tu App



La API



Base de Datos / Servidor



¿Qué hace?

Permite que dos sistemas distintos hablen entre sí sin saber cómo funciona el otro por dentro.

¿Por qué importa?

- **Integra:** Conecta funciones automáticamente (Ej: Loguearse con Google).
- **Eficiencia:** No reinventamos la rueda (Ej: Usar Stripe para cobrar). 💳
- **Innovación:** Creamos apps complejas rápido. 🗝️



# ¿Cómo se monetiza una API?

Los 4 modelos de negocio detrás de los datos.



## Suscripción

Tarifa fija mensual por acceso y límites de uso.  
Diferentes niveles según la necesidad.



## Pay-as-you-go

Pagás solo lo que consumís. Ideal para  
empresas que necesitan escalar flexiblemente.



## Por Transacción

Comisión pequeña por cada operación exitosa.  
Muy común en sistemas financieros.



## Freemium

Nivel gratuito básico + opciones premium pagas  
para enganchar a los desarrolladores.



# Las APIs en la vida salvaje 🌐

Quién usa qué modelo (Spoiler: los usás todos los días).



**AWS**

Rey del Pago por Uso.  
Pagás exactamente por el  
almacenamiento y cómputo  
que consumís en la nube.



**Twilio**

Pago por Uso o Suscripción.  
Te permite mandar SMS o  
videollamadas desde tu  
código sin comprar antenas.



**PayPal / Stripe**

Por Transacción.  
Resuelven la seguridad  
financiera a cambio de un  
porcentaje de cada venta  
exitosa.



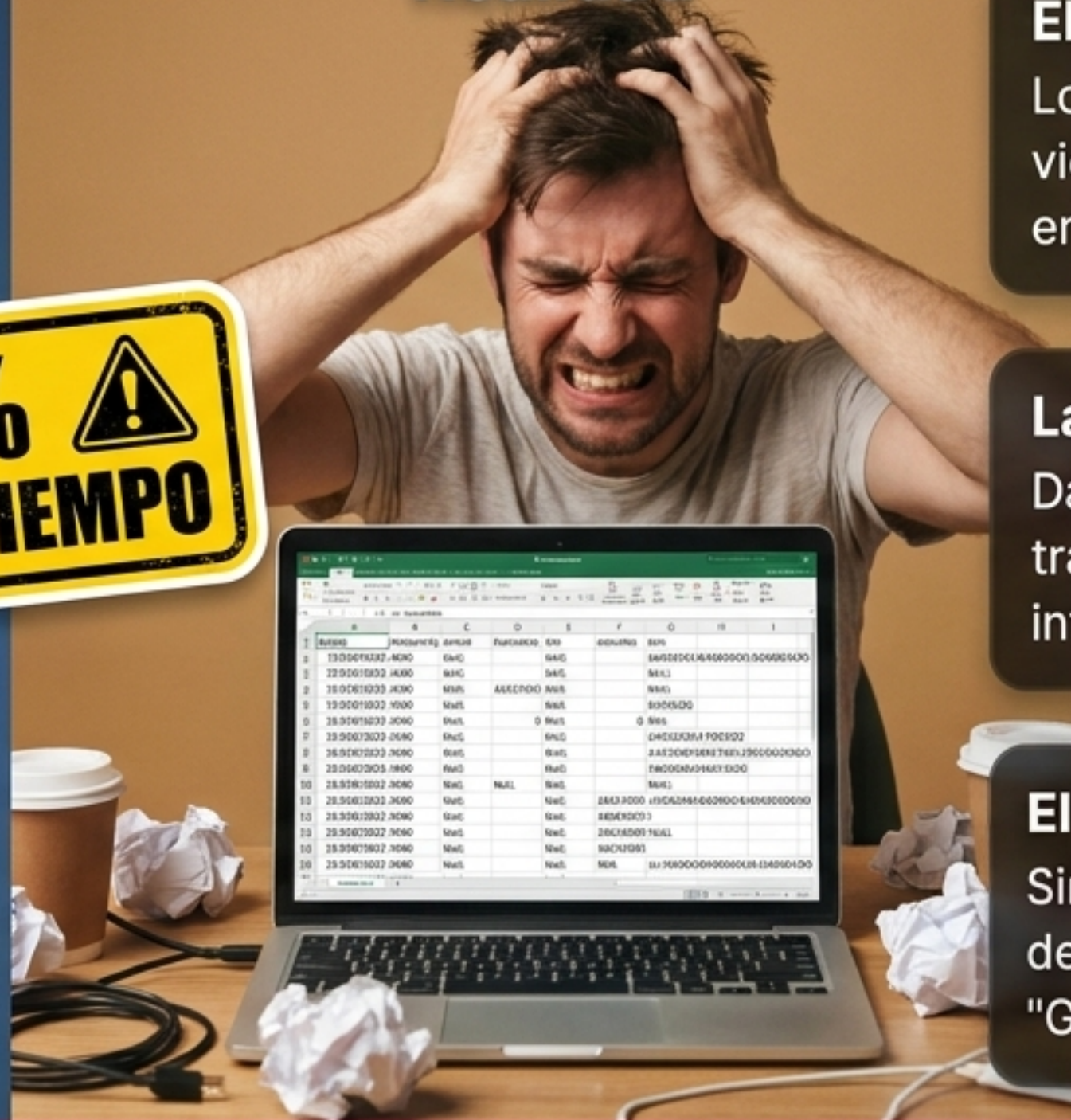
# Expectativa vs. Realidad de los Datos

Ya descargué los datos de la API... ¿y ahora?

## Expectativa



## Realidad



### El problema

Los datos en bruto (raw data) vienen desordenados, con errores, nulos o formatos raros.

### La solución

Data Wrangling. Limpiar, transformar y unificar la información.

### El Dato Duro

Sin buenos datos, los algoritmos de **Machine Learning** no sirven. "Garbage In, Garbage Out".



# El Pipeline de Wrangling (Parte 1)

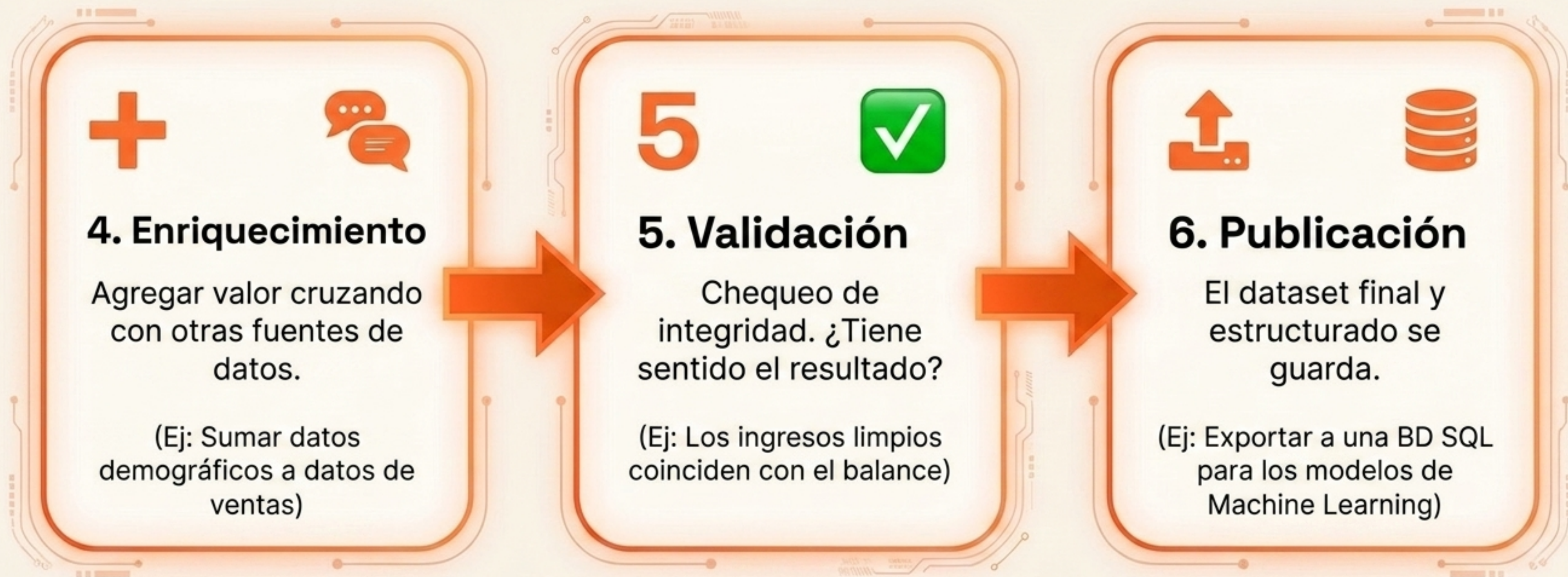
Las primeras tres fases para domar el dataset.





# El Pipeline de Wrangling (Parte 2) 🚀

Optimizando la data para el análisis final.





## Breakout: Misión Limpieza 🧼

5 minutos de debate en salas

### El Escenario:

Imagina que descargas ventas desde una API de PayPal y datos de clientes desde un viejo Excel.

### Preguntas a resolver con tu grupo:

1. ¿Qué problemas potenciales encontrarías en la etapa de **Descubrimiento** al mirar ambos archivos?
2. ¿De qué forma aplicarías el **Enriquecimiento** usando estos dos datasets?

**¡Un vocero por sala comparte al volver!**



# Combinando Fuerzas en Pandas 🐼

El corazón del enriquecimiento de datos.



- En Data Science, rara vez todo viene en un solo archivo.
- Tendremos CSVs locales, respuestas de APIs en la nube y bases de datos SQL separadas.
- **El objetivo:** Integrar múltiples fuentes para construir un dataset rico y completo.
- **Las herramientas estrella:** Las funciones `merge()` y `concat()` de Pandas.



# Merge vs. Concat 🥊

¿Cuándo usar cuál?

## 🔗 merge() (Cruce / Join)



- **Combina** filas basándose en claves comunes (Ej: ID\_Cliente).
- Similar al **JOIN** en SQL.
- **Expande** las columnas de tus datos cruzando información detallada de tablas distintas.

## 🧱 concat() (Apilado)



- **Apila DataFrames** a lo largo de un eje (filas o columnas).
- Útil para misma estructura, distintos periodos.
- (Ej: Apilar Ventas\_2022 + Ventas\_2023).



# ¿Qué está roto acá?

Desafío rápido visual en Pandas.



```
# Carga de datos
df_ventas = pd.DataFrame({'id': [1,2], 'monto': [100,200]})
df_clientes = pd.DataFrame({'id_cliente': [1,2], 'nombre': ['Ana', 'Sol']})

# Intentando cruzar los datos
df_final = pd.merge(df_ventas, df_clientes, on='id')
```

**¿Por qué fallará este código? (¡Escribí en el chat!)**



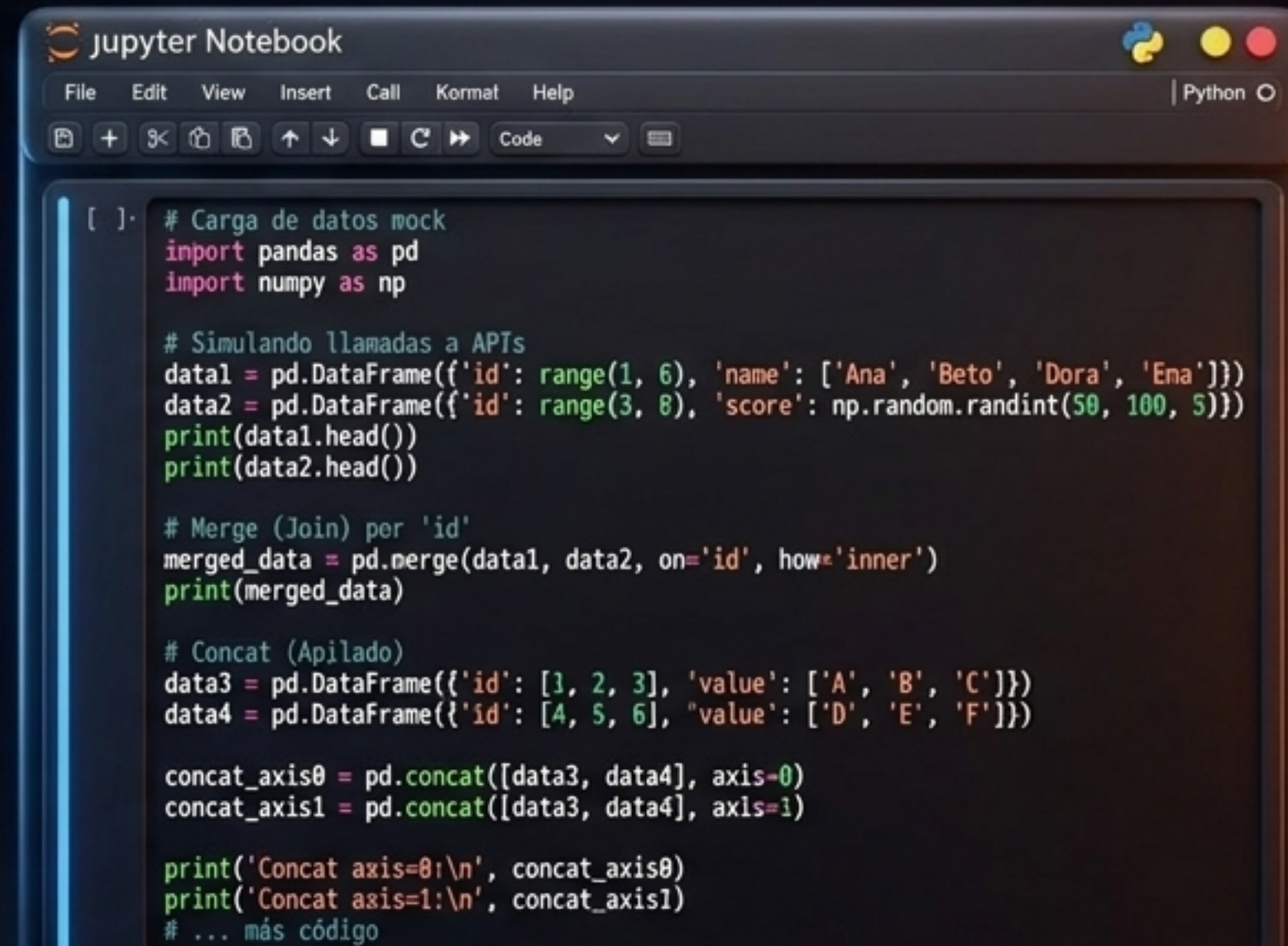
# Hands-on: Manos al teclado

Pasamos a la notebook.

Abre tu Jupyter Notebook de la clase.

Vamos a codear juntos:

- Importar datasets simulando llamadas a APIs.
- Aplicar **merge()** utilizando claves en común (on='id').
- Aplicar **concat()** para apilar en diferentes ejes (axis=0 y axis=1).



```
[ ]: # Carga de datos mock
import pandas as pd
import numpy as np

# Simulando llamadas a APIs
data1 = pd.DataFrame({'id': range(1, 6), 'name': ['Ana', 'Beto', 'Dora', 'Ena']})
data2 = pd.DataFrame({'id': range(3, 8), 'score': np.random.randint(50, 100, 5)})
print(data1.head())
print(data2.head())

# Merge (Join) per 'id'
merged_data = pd.merge(data1, data2, on='id', how='inner')
print(merged_data)

# Concat (Apilado)
data3 = pd.DataFrame({'id': [1, 2, 3], 'value': ['A', 'B', 'C']})
data4 = pd.DataFrame({'id': [4, 5, 6], 'value': ['D', 'E', 'F']})

concat_axis0 = pd.concat([data3, data4], axis=0)
concat_axis1 = pd.concat([data3, data4], axis=1)

print('Concat axis=0:\n', concat_axis0)
print('Concat axis=1:\n', concat_axis1)
# ... más código
```